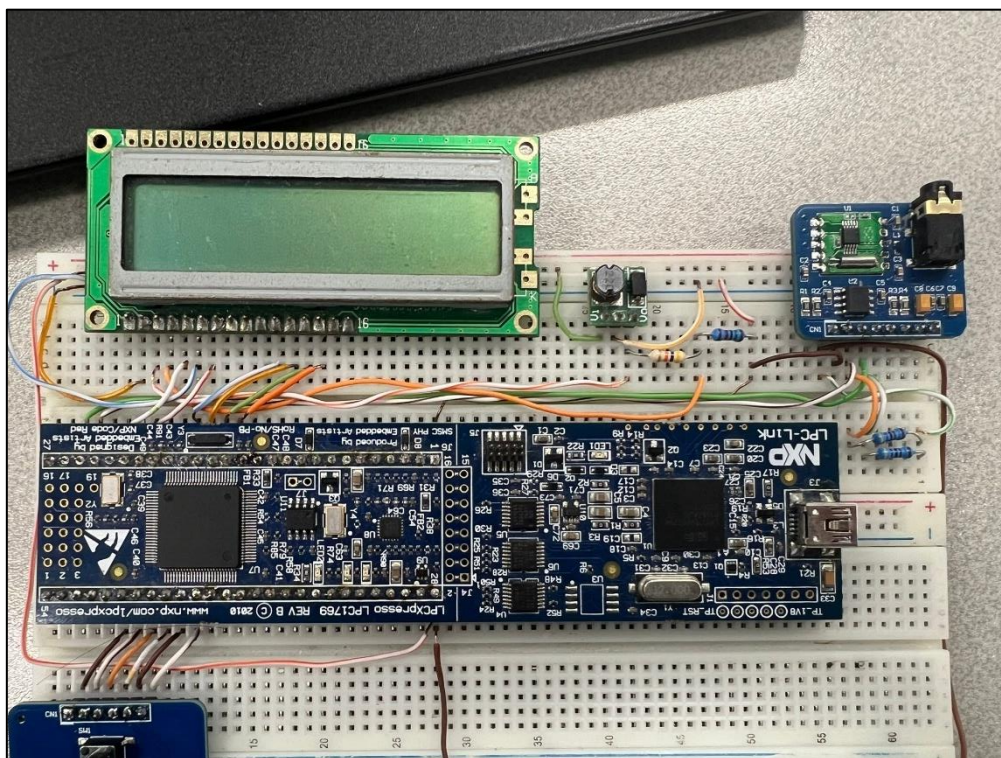


Sistemas Embebidos

Licenciatura em Engenharia Eletrônica e Telecomunicações e de Computadores



Trabalho realizado por:

Rodrigo Figueira Nº49347

Índice

1. Introdução.....	3
2. Arquitetura do sistema	3
1. Componentes de hardware.....	3
2. Diagrama de blocos	4
3. Funcionamento do sistema.....	4
1. Modo Normal (NORMAL_STATE).....	5
2. Modo Manutenção (MAINTENANCE_STATE)	6
4. Implementação de software	6
1. Estrutura do código	6
2. Comunicação com o módulo de rádio FM	7
3. Gestão do LCD e botões	7
4. Memória não volátil	8
5. Testes e resultados	8
6. Conclusão.....	9
 Figura 1- Diagrama de ligações	4
Figura 2 - Diagrama de estados.....	5
Figura 3 - Diagrama de camadas	7
Figura 4 - Modo Normal e Manutenção	8

1.Introdução

Durante a disciplina de Sistemas Embebidos, foi desenvolvido um sistema independente para sintonizar rádio FM, este integra funcionalidades de relógio e calendário, também tem a capacidade de guardar a estação e o volume selecionados.

O projeto teve como principal objetivo aplicar os conhecimentos adquiridos sobre microcontroladores, componentes, como se comunicam e a criação de uma boa interface para o utilizador.

2.Arquitetura do sistema

1. Componentes de hardware

O modelo elaborado foi construído seguindo uma lógica de peças encaixadas, com o microcontrolador atuar como a unidade central de processamento, gerindo a comunicação entre o teclado (entrada de dados) e o LCD e o rádio (saída de informações). Os principais periféricos usados foram:

- **Microcontrolador NXP LPC 1769:** Baseado na arquitetura ARM Cortex-M3, é o “coração” do sistema. É o encarregado pela execução da máquina de estados, gestão do protocolo I2C, controlar as portas GPIO, manutenção do relógio (RTC) e guardar dados persistentes na memória Flash interna (Setor 29).
- **Módulo de Rádio FM (RDA5807M):** Um recetor de rádio digital num único componente. A comunicação é realizada através do barramento I2C, permitindo definir a frequência, volume e configuração de áudio.
- **Display LCD 16x2 (HD44780):** O ecrã que o user vê. Foi configurado no modo de 4 bits para otimizar a utilização de pinos do microcontrolador.
- **Teclado 4x2 (Nav7Btn):** A interface de entrada composta por 4 linhas e 2 colunas. Com ele, é possível navegar nos menus, modificar configurações e gravar as preferências.
- **Step-Up:** Para alimentar o LCD com os 5V requeridos, foi preciso implementar um conversor Step-Up. Além disso, para assegurar que os caracteres fossem exibidos corretamente, ligou se o pino de contraste (VEE) ao divisor de tensão usado no conversor.

2. Diagrama de blocos

A figura seguinte demonstra as ligações físicas e lógicas dos componentes do sistema. O LPC1769 centraliza todas as operações, comunicando com o rádio via I2C e controlando o LCD e o teclado.

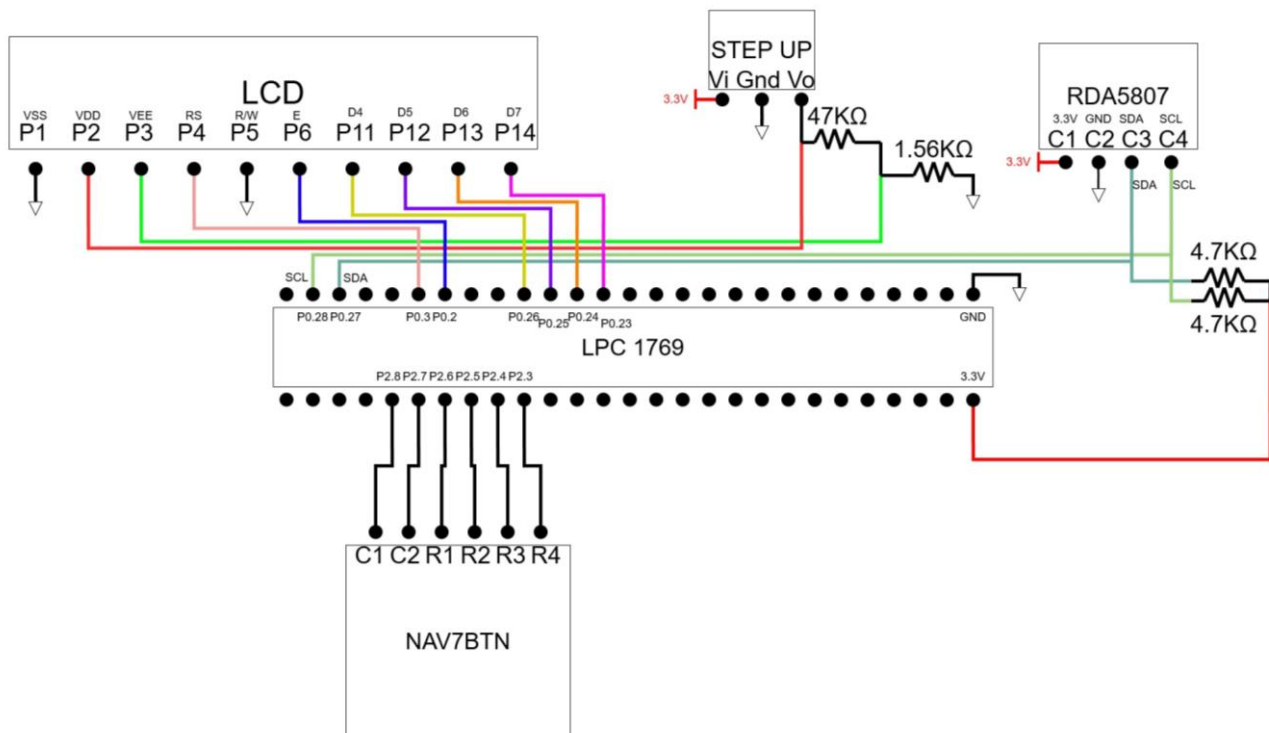


Figura 1- Diagrama de ligações

O sistema é alimentado via USB.

3. Funcionamento do sistema

O comportamento do sistema é gerido por uma máquina de estados, implementada no ficheiro statemachine.c. Ao iniciar, o sistema carrega configurações da memória não volátil e entra no “Modo Normal”. A transição entre modos e a forma como o utilizador interage são geridas através da leitura do teclado e atualização do display LCD constantemente.

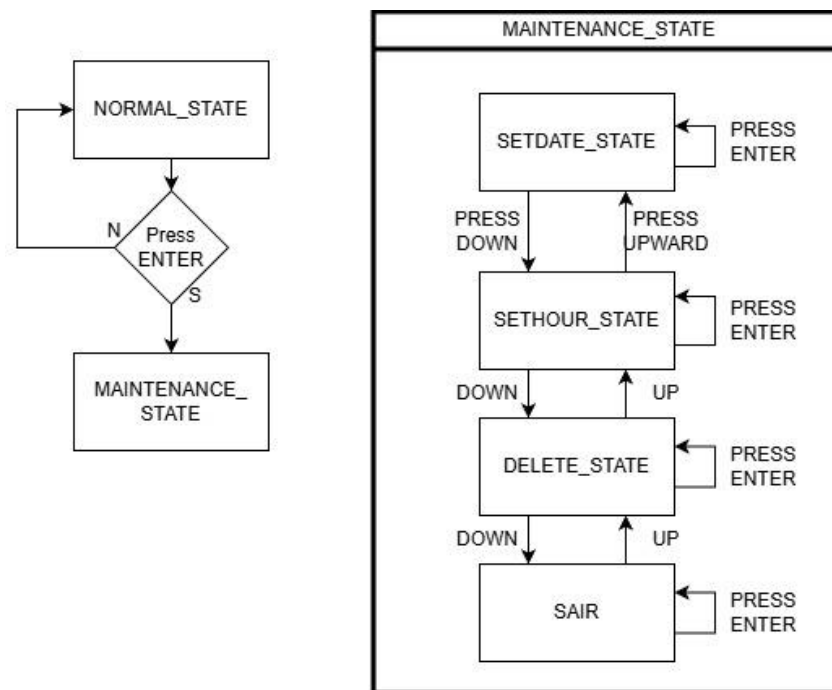


Figura 3 - Diagrama de estados

1. Modo Normal (NORMAL_STATE)

Este é o estado principal de uso, onde o sistema funciona como rádio e relógio. Aqui, o microcontrolador lê continuamente o RTC e as variáveis de controlo do rádio para mostrar informações no LCD em tempo real.

Interface visual:

- **Linha 1:** Apresenta a **data** (formato: DD/MM/AAAA) e o **volume** (de 0 a 15).
- **Linha 2:** Mostra a **hora** (HH:MM:SS) e a **frequência sintonizada** (MHz).

Interface e controles:

- **Teclas UP/DOWN:** Aumentam ou diminuem o volume.
- **Teclas LEFT/RIGHT:** Ajustam a frequência em passos de 0.1Mhz.
- **Teclas CENTER:** Executa a função de guardar definições. Inicia a escrita na memória flash, guardando a frequência e o volume para serem usados após um reinício.
- **Teclas ENTER:** Transita o sistema para o “Modo Manutenção”.

2. Modo Manutenção (MAINTENANCE_STATE)

O estado de manutenção atua como o menu principal de configuração do sistema. Ao entrar neste modo, a execução do rádio mantém-se em segundo plano, mas o display LCD altera-se para permitir a navegação entre submenus.

Lógica de navegação: Foi implementado um cursor virtual (>) que permite ao user seleccionar uma de quatro opções. As teclas **UP** e **DOWN** movem o cursor de forma rotativa entre as opções disponíveis:

- **Acertar data (SETDATE_STATE):** Permite a configuração do dia, do mês e do ano. Foi adicionado um algoritmo de verificação que ajusta automaticamente o dia máximo, dependendo do mês e do ano (anos bissextos).
- **Acertar hora (SETHOUR_STATE):** Permite a edição individual das horas, minutos e segundos. O sistema permite a navegação entre campos e valida os limites.
- **Apagar dados (DELETE_STATE):** Permite ao utilizador apagar as configurações armazenadas na memória flash, por segurança (e pedido no enunciado), este menu pede uma confirmação adicional ("ENTER" para apagar, "BACK" para retornar).
- **Voltar:** Regressa ao estado normal.

A escolha de qualquer submenu é confirmada através da tecla "ENTER", que altera a variável global de estado (CURRENT_STATE) para o modo correspondente.

4. Implementação de software

O software do sistema foi desenvolvido em linguagem C com uma arquitetura modular, utilizando MCUXpresso e as bibliotecas CMSIS para o microcontrolador LPC1769.

1. Estrutura do código

Para garantir a capacidade de se expandir e a facilidade de manutenção, foi adotado uma arquitetura em camadas:

- **Camada de drivers:** Agrupa ficheiros que interagem diretamente com o hardware (I2C.c, flash.c, lcd.c, ...).
- **Camada de lógica:** O ficheiro statemachine.c funciona como o elemento central do sistema. Simplifica a complexidade dos drivers e implementa a lógica da máquina de estados, gerindo as mudanças entre menus e a resposta aos comandos do utilizador.
- **Camada de aplicação:** O ficheiro projeto.c é responsável apenas pela inicialização dos periféricos e pelo loop infinito que realiza a execução da máquina de estados.

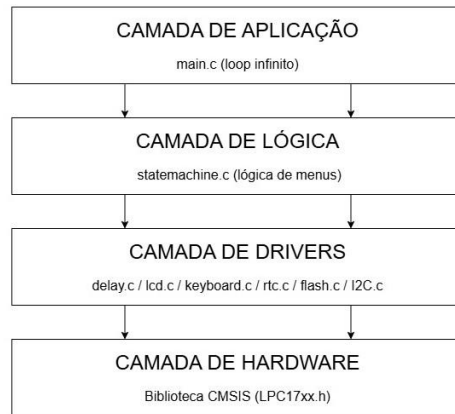


Figura 5 - Diagrama de camadas

Esta divisão permite que, por exemplo, substituir o driver do LCD sem precisar refazer a lógica central do programa.

2. Comunicação com o módulo de rádio FM

A conexão com o chip RDA5807M é realizada através do barramento I2C. O microcontrolador age como “Master” e o rádio como “Slave”.

Foi implementado um driver I2C via registos do LPC (I2C0), ajustado para uma frequência de 100Khz, a comunicação utiliza o modo de acesso sequencial (endereço 0x10), que permite alterar todos os registos internos do rádio numa única transação escrita, melhorando o desempenho.

O software transforma a frequência visível para o formato de 10 bits necessário ao chip pela fórmula: $Canal = \frac{Freq(Mhz) - 87.0}{0.1}$, este valor é dividido em dois bytes (High e Low) e enviado no payload I2C junto com os bits de controlo “TUNE” e “ENABLE”.

3. Gestão do LCD e botões

- **LCD (Modo 4 bits):** Para economizar o uso dos pinos do microcontrolador, o driver do LCD foi feito em modo de 4 bits. Isto implica que cada byte de comando ou dados é enviado em dois “nibbles” separados. Criou-se a função LCDText_Printf que permite formatar strings diretamente, o que facilita amostrar a data e a hora.
- **Teclado matricial:** A leitura dos botões é feita através de uma técnica de varrimento. O software coloca sequencialmente cada linha a nível lógico baixo (0V) e lê o estado das colunas, se uma coluna estiver em nível baixo, identifica-se qual tecla foi pressionada pelo cruzamento Linha/Coluna.
Foi implementado uma lógica simples de verificação de estado, NAVBTN_Pressed, que garante que uma tecla só é registada quando o seu estado se altera, prevenindo múltiplas leituras erradas.

4. Memória não volátil

A capacidade de manter dados (guardar volume e frequência) usa a tecnologia IAP do LPC 1769, que permite escrever na memória flash durante a execução do programa.

Foi escolhido o setor 29 da flash para evitar conflitos com o código do programa. Foi definida uma struct contendo uma “marca pessoal” (para poder verificar se há dados guardados ou não), a frequência e o volume. Ao iniciar o sistema verifica se existe a marca pessoal, se não, carrega valores padrão.

5. Testes e resultados

Os testes feitos no laboratório demonstraram a estabilidade e a funcionalidade do sistema criado. Após a inicialização, verificou-se que a interface no LCD exibe os dados de forma clara, sem falhas visuais, o que facilita a leitura instantânea da data, hora, frequência e volume.

A transição entre o modo normal e o modo de manutenção ocorre de forma fluida através do teclado matricial, o que aprova a lógica utilizada na máquina de estados.

Em relação ao controle do rádio, o módulo RDA5807M respondeu sem demora a todos os comandos enviados via protocolo I2C. Foi possível confirmar que as teclas de navegação alteram a frequência em passos de 0.1Mhz e ajustam o volume como esperado, adicionalmente a funcionalidade de persistência de dados foi validada com sucesso; ao salvar as configurações e reiniciar o fornecimento de energia da placa, o sistema restaurou automaticamente a última estação e volume guardados.

O RTC manteve a contagem do tempo precisa, e os algoritmos de verificação de data impediram a introdução de dias inválidos nos meses de menor duração.



Figura 7 - Modo Normal e Manutenção

6. Conclusão

Este trabalho resultou no desenvolvimento bem-sucedido de um controlador de rádio FM, atendendo a todos os critérios técnicos e funcionais propostos no projeto.

A realização deste projeto ajudou a solidificar conhecimentos práticos importantes na área dos sistemas embebidos, com destaque para a importância da arquitetura de software modular e o uso de máquinas de estados finitos para a gestão de interfaces do user.

Em resumo, o protótipo final funciona de forma estável, apresentando uma solução completa para a sintonização e gestão de rádio FM com persistência de dados.